

# Introduction to Neural Networks: Bank Churn prediction

## Problem Statement

### Context

Businesses like banks which provide service have to worry about problem of 'Customer Churn' i.e. customers leaving and joining another service provider. It is important to understand which aspects of the service influence a customer's decision in this regard. Management can concentrate efforts on improvement of service, keeping in mind these priorities.

### Objective

You as a Data scientist with the bank need to build a neural network based classifier that can determine whether a customer will leave the bank or not in the next 6 months.

### Data Dictionary:

- CustomerId: Unique ID which is assigned to each customer
- Surname: Last name of the customer
- CreditScore: It defines the credit history of the customer.
- Geography: A customer's location
- Gender: It defines the Gender of the customer
- Age: Age of the customer
- Tenure: Number of years for which the customer has been with the bank
- NumOfProducts: refers to the number of products that a customer has purchased through the bank.
- Balance: Account balance
- HasCrCard: It is a categorical variable which decides whether the customer has credit card or not.
- EstimatedSalary: Estimated salary
- isActiveMember: Is is a categorical variable which decides whether the customer is active member of the bank or not ( Active member in the sense, using bank products regularly, making transactions etc )
- Exited : whether or not the customer left the bank within six month. It can take two values **0=No ( Customer did not leave the bank )** 1=Yes ( Customer left the bank )

## Importing necessary libraries

```
In [47]: import pandas as pd
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
import io
from sklearn.preprocessing import StandardScaler
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import tensorflow as tf
from sklearn.metrics import confusion_matrix
```

## Loading the dataset

```
In [3]: from google.colab import drive
drive.mount('/content/drive')
# read the data
df = pd.read_csv('/content/drive/MyDrive/Churn.csv')
```

Mounted at /content/drive

# Data Overview

```
In [4]: # Checking the number of rows and columns in the data
df.shape
```

```
Out[4]: (10000, 14)
```

```
In [5]: # let's view the first 5 rows of the data
df.head()
```

```
Out[5]:
```

|   | RowNumber | CustomerId | Surname  | CreditScore | Geography | Gender | Age | Tenure | Balance   | NumOfProducts | HasCrCard | IsActiveMember |
|---|-----------|------------|----------|-------------|-----------|--------|-----|--------|-----------|---------------|-----------|----------------|
| 0 | 1         | 15634602   | Hargrave | 619         | France    | Female | 42  | 2      | 0.00      | 1             | 1         |                |
| 1 | 2         | 15647311   | Hill     | 608         | Spain     | Female | 41  | 1      | 83807.86  | 1             | 0         |                |
| 2 | 3         | 15619304   | Onio     | 502         | France    | Female | 42  | 8      | 159660.80 | 3             | 1         |                |
| 3 | 4         | 15701354   | Boni     | 699         | France    | Female | 39  | 1      | 0.00      | 2             | 0         |                |
| 4 | 5         | 15737888   | Mitchell | 850         | Spain     | Female | 43  | 2      | 125510.82 | 1             | 1         |                |

```
In [6]: # let's check the data types of the columns in the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   RowNumber              10000 non-null  int64
1   CustomerId             10000 non-null  int64
2   Surname                10000 non-null  object
3   CreditScore            10000 non-null  int64
4   Geography              10000 non-null  object
5   Gender                 10000 non-null  object
6   Age                    10000 non-null  int64
7   Tenure                 10000 non-null  int64
8   Balance                10000 non-null  float64
9   NumOfProducts          10000 non-null  int64
10  HasCrCard              10000 non-null  int64
11  IsActiveMember         10000 non-null  int64
12  EstimatedSalary        10000 non-null  float64
13  Exited                 10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
In [7]: # let's check for duplicate values in the data
df.duplicated().sum()
```

```
Out[7]: 0
```

```
In [8]: # let's check for missing values in the data
round(df.isnull().sum() / df.isnull().count() * 100, 2)
```

```
Out[8]: RowNumber      0.0
CustomerId    0.0
Surname       0.0
CreditScore   0.0
Geography     0.0
Gender        0.0
Age           0.0
Tenure        0.0
Balance       0.0
NumOfProducts 0.0
HasCrCard     0.0
IsActiveMember 0.0
EstimatedSalary 0.0
Exited        0.0
dtype: float64
```

```
In [9]: # print uniques
df.nunique()
```

```
Out[9]: RowNumber      10000
CustomerId    10000
Surname       2932
CreditScore    460
Geography      3
Gender         2
Age           70
Tenure        11
Balance       6382
NumOfProducts    4
HasCrCard       2
IsActiveMember   2
EstimatedSalary 9999
Exited         2
dtype: int64
```

```
In [10]: df=df.drop(['RowNumber','CustomerId','Surname'],axis=1)
df.head()
```

```
Out[10]:
```

|   | CreditScore | Geography | Gender | Age | Tenure | Balance   | NumOfProducts | HasCrCard | IsActiveMember | EstimatedSalary | Exited |
|---|-------------|-----------|--------|-----|--------|-----------|---------------|-----------|----------------|-----------------|--------|
| 0 | 619         | France    | Female | 42  | 2      | 0.00      | 1             | 1         | 1              | 101348.88       | 1      |
| 1 | 608         | Spain     | Female | 41  | 1      | 83807.86  | 1             | 0         | 1              | 112542.58       | 0      |
| 2 | 502         | France    | Female | 42  | 8      | 159660.80 | 3             | 1         | 0              | 113931.57       | 1      |
| 3 | 699         | France    | Female | 39  | 1      | 0.00      | 2             | 0         | 0              | 93826.63        | 0      |
| 4 | 850         | Spain     | Female | 43  | 2      | 125510.82 | 1             | 1         | 1              | 79084.10        | 0      |

```
In [11]: # let's view the statistical summary of the numerical columns in the data
df.describe().T
```

```
Out[11]:
```

|                 | count   | mean          | std          | min    | 25%      | 50%        | 75%         | max       |
|-----------------|---------|---------------|--------------|--------|----------|------------|-------------|-----------|
| CreditScore     | 10000.0 | 650.528800    | 96.653299    | 350.00 | 584.00   | 652.000    | 718.0000    | 850.00    |
| Age             | 10000.0 | 38.921800     | 10.487806    | 18.00  | 32.00    | 37.000     | 44.0000     | 92.00     |
| Tenure          | 10000.0 | 5.012800      | 2.892174     | 0.00   | 3.00     | 5.000      | 7.0000      | 10.00     |
| Balance         | 10000.0 | 76485.889288  | 62397.405202 | 0.00   | 0.00     | 97198.540  | 127644.2400 | 250898.09 |
| NumOfProducts   | 10000.0 | 1.530200      | 0.581654     | 1.00   | 1.00     | 1.000      | 2.0000      | 4.00      |
| HasCrCard       | 10000.0 | 0.705500      | 0.455840     | 0.00   | 0.00     | 1.000      | 1.0000      | 1.00      |
| IsActiveMember  | 10000.0 | 0.515100      | 0.499797     | 0.00   | 0.00     | 1.000      | 1.0000      | 1.00      |
| EstimatedSalary | 10000.0 | 100090.239881 | 57510.492818 | 11.58  | 51002.11 | 100193.915 | 149388.2475 | 199992.48 |
| Exited          | 10000.0 | 0.203700      | 0.402769     | 0.00   | 0.00     | 0.000      | 0.0000      | 1.00      |

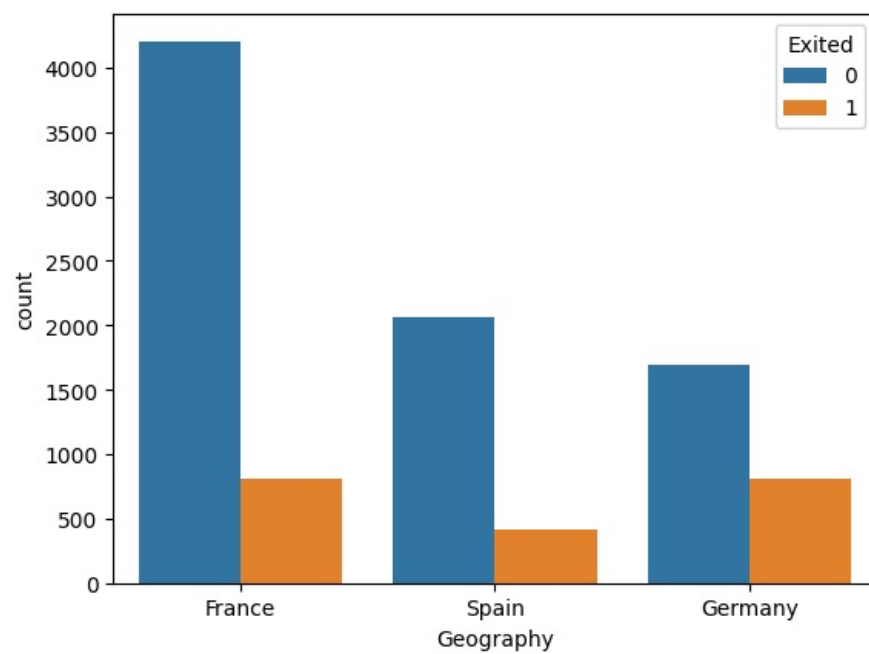
## Exploratory Data Analysis

- EDA is an important part of any project involving data.
- It is important to investigate and understand the data better before building a model with it.
- A few questions have been mentioned below which will help you approach the analysis in the right manner and generate insights from the data.
- A thorough analysis of the data, in addition to the questions mentioned below, should be done.

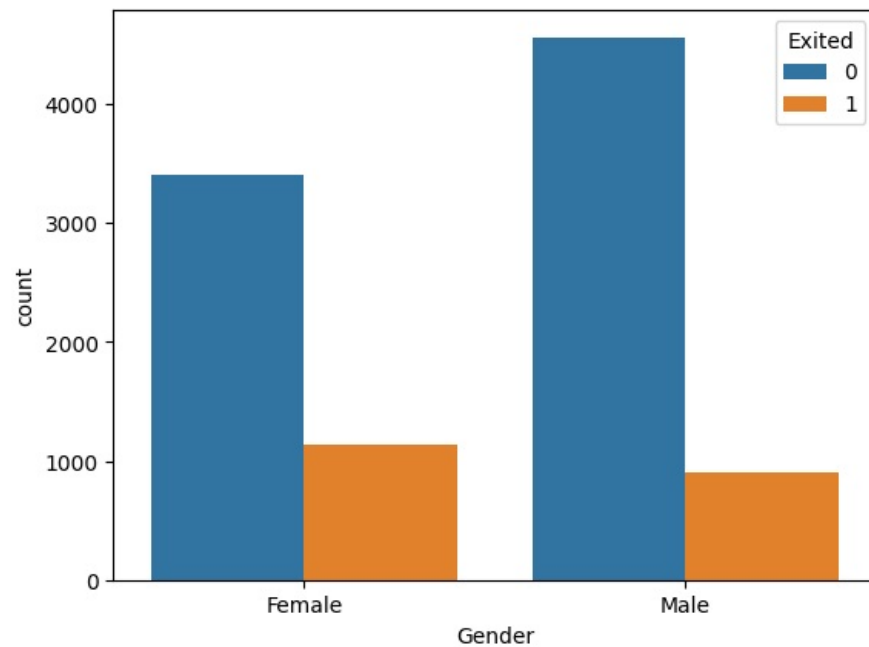
### Questions:

1. What is the distribution of the credit score of customers? Are there any noticeable patterns or outliers in the distribution?
2. How many active members are there with the bank?
3. How are the different customer attributes correlated to each other?
4. Who is churning more when compared to males and females?
5. Customers from which geographical part are churning more?

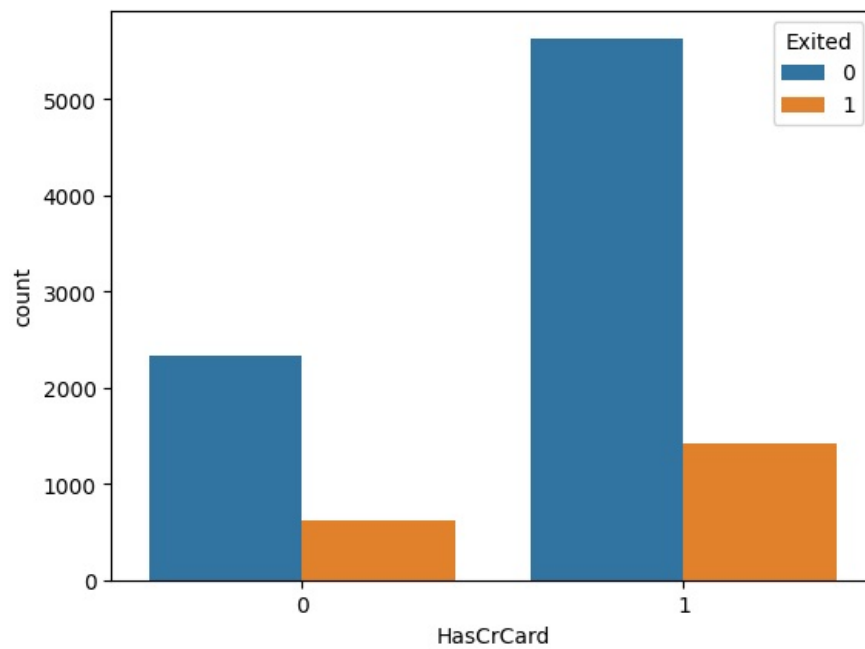
```
In [13]: sns.countplot(x='Geography',data=df,hue='Exited')
plt.show()
```



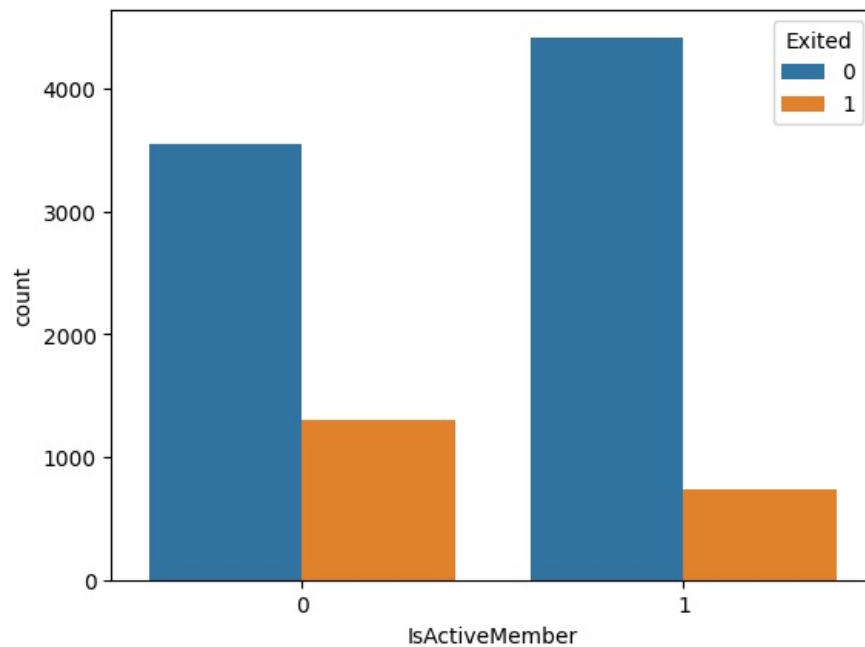
```
In [14]: sns.countplot(x='Gender',data=df,hue='Exited')  
plt.show()
```



```
In [15]: sns.countplot(x='HasCrCard',data=df,hue='Exited')  
plt.show()
```



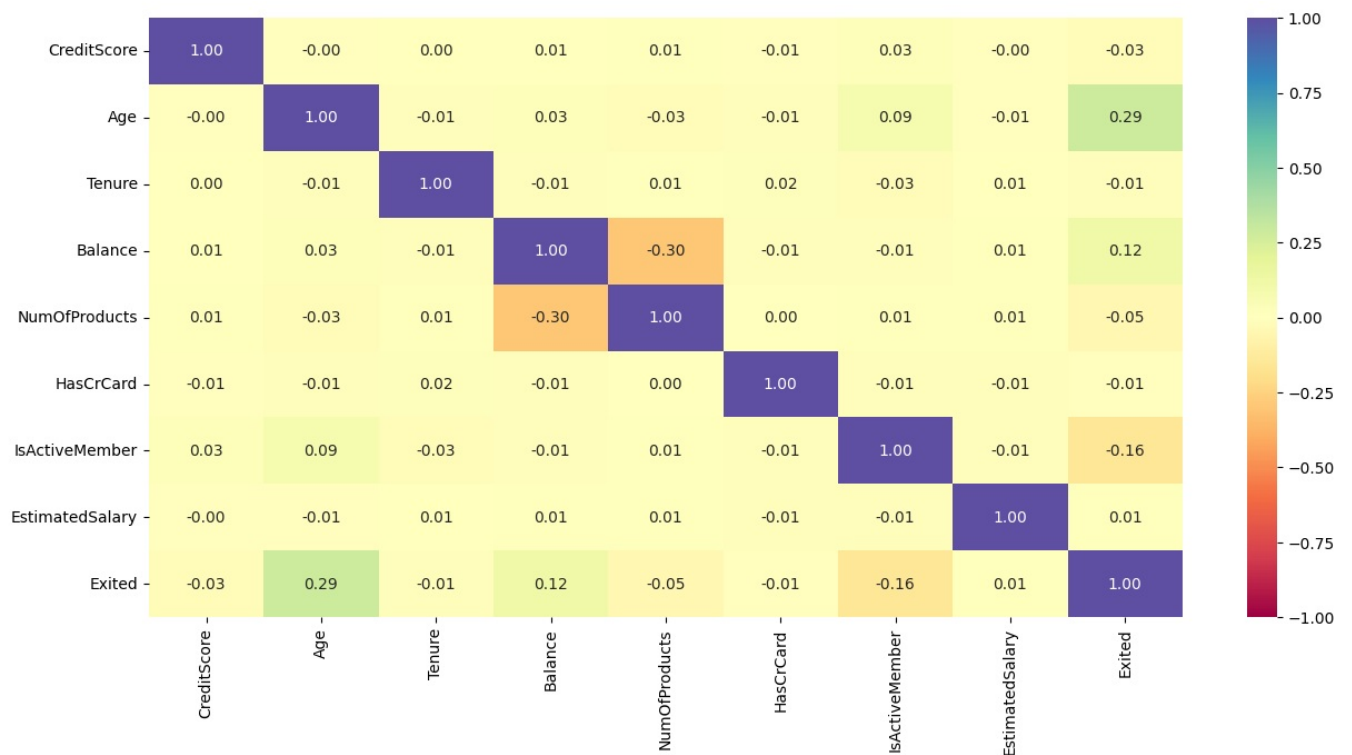
```
In [16]: sns.countplot(x='IsActiveMember',data=df,hue='Exited')
plt.show()
```



```
In [23]: plt.figure(figsize=(15, 7))
sns.heatmap(df.corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral")
plt.show()
```

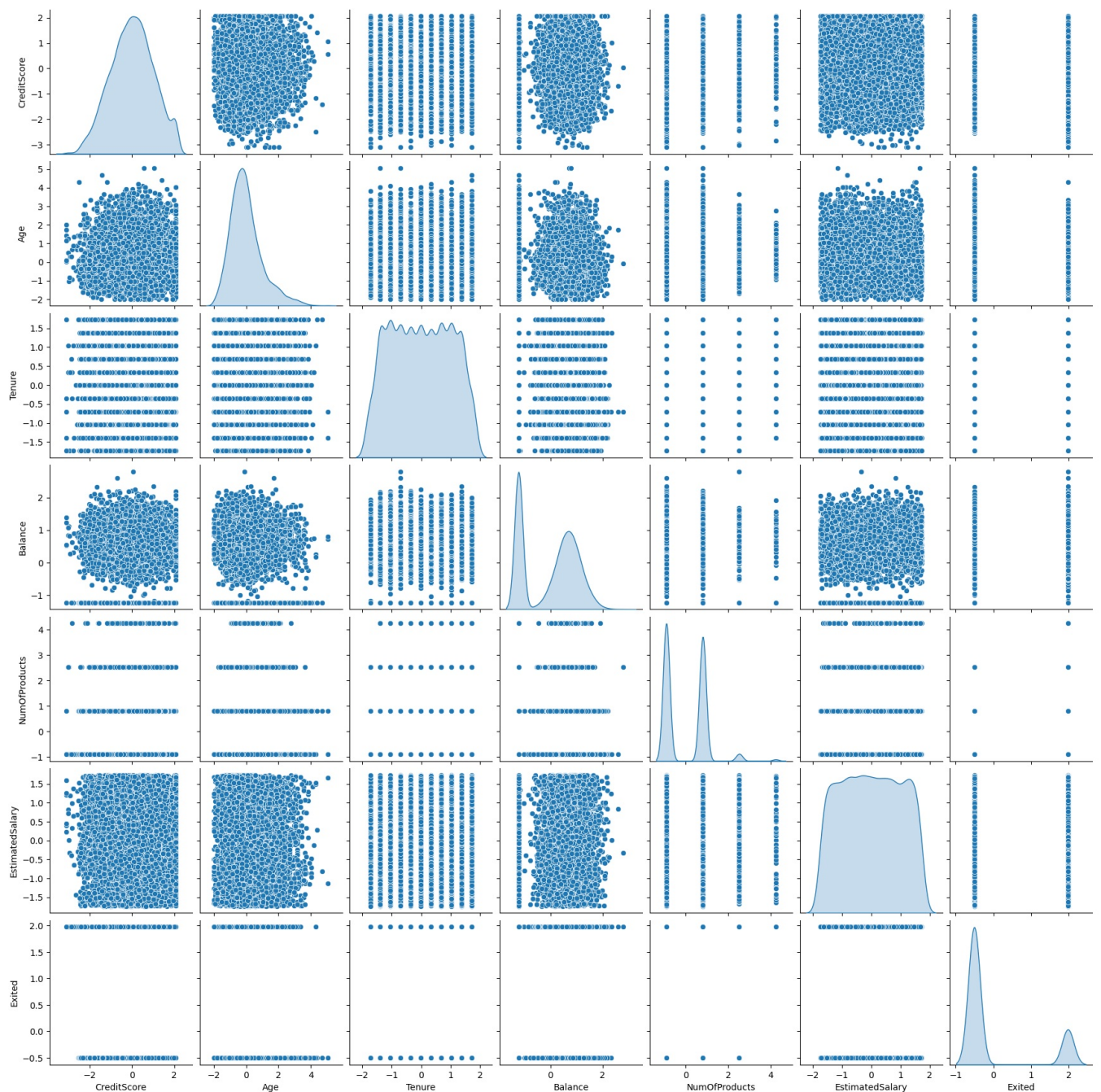
<ipython-input-23-d207b7d46160>:2: FutureWarning: The default value of numeric\_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric\_only to silence this warning.

```
sns.heatmap(df.corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral")
```



```
In [24]: scaler=StandardScaler()
subset=df.drop(['Geography','Gender','HasCrCard','IsActiveMember'],axis=1)
scaled=scaler.fit_transform(subset)
scaled_df=pd.DataFrame(scaled,columns=subset.columns)
sns.pairplot(scaled_df,diag_kind='kde')
```

```
Out[24]: <seaborn.axisgrid.PairGrid at 0x7eb54ebcb940>
```



- Most of the customers are located in France.
- There are more male customers than female customers.
- Most of the customers have been with the bank for less than a year.

- 'Age' and 'Exited' are highly correlated.
- 'Number of Products' and 'Exited' have the highest negative correlation.

## Data Preprocessing

- Missing value treatment
- Feature engineering (if needed)
- Outlier detection and treatment (if needed)
- Preparing data for modeling
- Any other preprocessing steps (if needed)

Distinguishing the Target and Feature Set & dividing dataset into Training and Test sets

```
In [25]: X=df.drop('Exited',axis=1)
y=df.pop('Exited')
```

```
In [26]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.10,random_state=5)
X_train,X_val,y_train,y_val=train_test_split(X_train,y_train,test_size=0.10,random_state=5)
print("X_train size is {}".format(X_train.shape[0]))
print("X_val size is {}".format(X_val.shape[0]))
print("X_test size is {}".format(X_test.shape[0]))

X_train size is 8100
X_val size is 900
X_test size is 1000
```

```
In [30]: scaler=StandardScaler()
num_cols=['CreditScore','Age','Tenure','Balance','NumOfProducts','EstimatedSalary']
num_subset=scaler.fit_transform(X_train[num_cols])
X_train_num_df=pd.DataFrame(num_subset,columns=num_cols)
X_train_num_df['Geography']=list(X_train['Geography'])
X_train_num_df['Gender']=list(X_train['Gender'])
X_train_num_df['HasCrCard']=list(X_train['HasCrCard'])
X_train_num_df['IsActiveMember']=list(X_train['IsActiveMember'])
X_train_num_df.head()

num_subset=scaler.fit_transform(X_val[num_cols])
X_val_num_df=pd.DataFrame(num_subset,columns=num_cols)
X_val_num_df['Geography']=list(X_val['Geography'])
X_val_num_df['Gender']=list(X_val['Gender'])
X_val_num_df['HasCrCard']=list(X_val['HasCrCard'])
X_val_num_df['IsActiveMember']=list(X_val['IsActiveMember'])
num_subset=scaler.fit_transform(X_test[num_cols])
X_test_num_df=pd.DataFrame(num_subset,columns=num_cols)
X_test_num_df['Geography']=list(X_test['Geography'])
X_test_num_df['Gender']=list(X_test['Gender'])
X_test_num_df['HasCrCard']=list(X_test['HasCrCard'])
X_test_num_df['IsActiveMember']=list(X_test['IsActiveMember'])
```

```
In [31]: # Converting the categorical features into numerical
X_train_num_df=pd.get_dummies(X_train_num_df,columns=['Geography','Gender'])
X_test_num_df=pd.get_dummies(X_test_num_df,columns=['Geography','Gender'])
X_val_num_df=pd.get_dummies(X_val_num_df,columns=['Geography','Gender'])
X_train_num_df.head()
```

```
Out[31]:
```

|   | CreditScore | Age       | Tenure    | Balance   | NumOfProducts | EstimatedSalary | HasCrCard | IsActiveMember | Geography_France | Geography |
|---|-------------|-----------|-----------|-----------|---------------|-----------------|-----------|----------------|------------------|-----------|
| 0 | -1.178587   | -1.041960 | -1.732257 | 0.198686  | 0.820905      | 1.560315        | 1         | 1              | 1                |           |
| 1 | -0.380169   | -1.326982 | 1.730718  | -0.022020 | -0.907991     | -0.713592       | 1         | 0              | 0                |           |
| 2 | -0.349062   | 1.808258  | -0.693364 | 0.681178  | 0.820905      | -1.126515       | 1         | 0              | 0                |           |
| 3 | 0.625629    | 2.378302  | -0.347067 | -1.229191 | 0.820905      | -1.682740       | 1         | 1              | 1                |           |
| 4 | -0.203895   | -1.136967 | 1.730718  | 0.924256  | -0.907991     | 1.332535        | 1         | 1              | 0                |           |

## Model Building

### Model Evaluation Criterion

- Initialise and build the Model

### Model Building: Neural Network

Model Building: Neural Network with TensorFlow

Model Improvement: Neural Network model with Dropout

Model Building: Neural Network model with Adam Optimizer

Model Improvement: Neural Network model with Hyperparameter tuning

Model Improvement: Neural Network model with balanced data

## Final Model

```
In [34]: model=Sequential()  
model.add(Dense(7,activation='relu'))  
model.add(Dense(10,activation='relu'))  
model.add(Dense(1,activation='sigmoid'))
```

```
In [38]: optimizer=tf.keras.optimizers.Adam(0.01)  
model.compile(loss='binary_crossentropy',optimizer=optimizer,metrics=['accuracy'])
```

```
In [39]: model.fit(X_train_num_df,y_train,epochs=100,batch_size=10,verbose=1)
```

```
Epoch 1/100  
810/810 [=====] - 3s 2ms/step - loss: 0.4065 - accuracy: 0.8206  
Epoch 2/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3588 - accuracy: 0.8520  
Epoch 3/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3482 - accuracy: 0.8584  
Epoch 4/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3466 - accuracy: 0.8583  
Epoch 5/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3437 - accuracy: 0.8567  
Epoch 6/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3420 - accuracy: 0.8596  
Epoch 7/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3431 - accuracy: 0.8630  
Epoch 8/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3430 - accuracy: 0.8620  
Epoch 9/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3408 - accuracy: 0.8625  
Epoch 10/100  
810/810 [=====] - 4s 5ms/step - loss: 0.3415 - accuracy: 0.8616  
Epoch 11/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3376 - accuracy: 0.8635  
Epoch 12/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3386 - accuracy: 0.8614  
Epoch 13/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3395 - accuracy: 0.8626  
Epoch 14/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3386 - accuracy: 0.8617  
Epoch 15/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3377 - accuracy: 0.8627  
Epoch 16/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3372 - accuracy: 0.8627  
Epoch 17/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3369 - accuracy: 0.8604  
Epoch 18/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3366 - accuracy: 0.8643  
Epoch 19/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3369 - accuracy: 0.8625  
Epoch 20/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3345 - accuracy: 0.8625  
Epoch 21/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3348 - accuracy: 0.8621  
Epoch 22/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3336 - accuracy: 0.8636  
Epoch 23/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3347 - accuracy: 0.8646  
Epoch 24/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3344 - accuracy: 0.8664  
Epoch 25/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3345 - accuracy: 0.8664  
Epoch 26/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3329 - accuracy: 0.8654  
Epoch 27/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3344 - accuracy: 0.8641  
Epoch 28/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3330 - accuracy: 0.8663  
Epoch 29/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3333 - accuracy: 0.8656  
Epoch 30/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3319 - accuracy: 0.8658  
Epoch 31/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3332 - accuracy: 0.8630  
Epoch 32/100
```

810/810 [=====] - 2s 2ms/step - loss: 0.3323 - accuracy: 0.8642  
Epoch 33/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3342 - accuracy: 0.8628  
Epoch 34/100  
810/810 [=====] - 3s 4ms/step - loss: 0.3333 - accuracy: 0.8632  
Epoch 35/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3314 - accuracy: 0.8644  
Epoch 36/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3318 - accuracy: 0.8638  
Epoch 37/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3321 - accuracy: 0.8658  
Epoch 38/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3304 - accuracy: 0.8646  
Epoch 39/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3336 - accuracy: 0.8638  
Epoch 40/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3326 - accuracy: 0.8631  
Epoch 41/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3312 - accuracy: 0.8678  
Epoch 42/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3312 - accuracy: 0.8631  
Epoch 43/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3303 - accuracy: 0.8646  
Epoch 44/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3329 - accuracy: 0.8617  
Epoch 45/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3320 - accuracy: 0.8631  
Epoch 46/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3316 - accuracy: 0.8644  
Epoch 47/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3316 - accuracy: 0.8643  
Epoch 48/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3311 - accuracy: 0.8640  
Epoch 49/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3309 - accuracy: 0.8638  
Epoch 50/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3293 - accuracy: 0.8668  
Epoch 51/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3322 - accuracy: 0.8637  
Epoch 52/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3321 - accuracy: 0.8621  
Epoch 53/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3317 - accuracy: 0.8638  
Epoch 54/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3305 - accuracy: 0.8640  
Epoch 55/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3300 - accuracy: 0.8656  
Epoch 56/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3316 - accuracy: 0.8599  
Epoch 57/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3308 - accuracy: 0.8633  
Epoch 58/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3315 - accuracy: 0.8619  
Epoch 59/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3315 - accuracy: 0.8635  
Epoch 60/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3302 - accuracy: 0.8656  
Epoch 61/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3301 - accuracy: 0.8656  
Epoch 62/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3292 - accuracy: 0.8626  
Epoch 63/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3290 - accuracy: 0.8658  
Epoch 64/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3307 - accuracy: 0.8657  
Epoch 65/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3283 - accuracy: 0.8649  
Epoch 66/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3292 - accuracy: 0.8658  
Epoch 67/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3289 - accuracy: 0.8672  
Epoch 68/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3304 - accuracy: 0.8656  
Epoch 69/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3293 - accuracy: 0.8636  
Epoch 70/100  
810/810 [=====] - 3s 3ms/step - loss: 0.3279 - accuracy: 0.8644  
Epoch 71/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3283 - accuracy: 0.8649  
Epoch 72/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3290 - accuracy: 0.8660  
Epoch 73/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3311 - accuracy: 0.8664  
Epoch 74/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3307 - accuracy: 0.8638  
Epoch 75/100  
810/810 [=====] - 2s 2ms/step - loss: 0.3308 - accuracy: 0.8640  
Epoch 76/100  
810/810 [=====] - 2s 3ms/step - loss: 0.3310 - accuracy: 0.8626

```

Epoch 77/100
810/810 [=====] - 2s 3ms/step - loss: 0.3278 - accuracy: 0.8658
Epoch 78/100
810/810 [=====] - 2s 3ms/step - loss: 0.3305 - accuracy: 0.8637
Epoch 79/100
810/810 [=====] - 2s 3ms/step - loss: 0.3296 - accuracy: 0.8647
Epoch 80/100
810/810 [=====] - 3s 3ms/step - loss: 0.3281 - accuracy: 0.8643
Epoch 81/100
810/810 [=====] - 2s 3ms/step - loss: 0.3287 - accuracy: 0.8679
Epoch 82/100
810/810 [=====] - 2s 2ms/step - loss: 0.3297 - accuracy: 0.8669
Epoch 83/100
810/810 [=====] - 2s 2ms/step - loss: 0.3266 - accuracy: 0.8654
Epoch 84/100
810/810 [=====] - 2s 2ms/step - loss: 0.3273 - accuracy: 0.8674
Epoch 85/100
810/810 [=====] - 2s 2ms/step - loss: 0.3280 - accuracy: 0.8660
Epoch 86/100
810/810 [=====] - 2s 3ms/step - loss: 0.3294 - accuracy: 0.8647
Epoch 87/100
810/810 [=====] - 2s 2ms/step - loss: 0.3283 - accuracy: 0.8648
Epoch 88/100
810/810 [=====] - 2s 3ms/step - loss: 0.3296 - accuracy: 0.8646
Epoch 89/100
810/810 [=====] - 2s 3ms/step - loss: 0.3326 - accuracy: 0.8627
Epoch 90/100
810/810 [=====] - 2s 3ms/step - loss: 0.3289 - accuracy: 0.8652
Epoch 91/100
810/810 [=====] - 2s 2ms/step - loss: 0.3273 - accuracy: 0.8649
Epoch 92/100
810/810 [=====] - 2s 2ms/step - loss: 0.3310 - accuracy: 0.8654
Epoch 93/100
810/810 [=====] - 2s 2ms/step - loss: 0.3310 - accuracy: 0.8625
Epoch 94/100
810/810 [=====] - 2s 2ms/step - loss: 0.3273 - accuracy: 0.8683
Epoch 95/100
810/810 [=====] - 2s 2ms/step - loss: 0.3293 - accuracy: 0.8664
Epoch 96/100
810/810 [=====] - 2s 2ms/step - loss: 0.3289 - accuracy: 0.8665
Epoch 97/100
810/810 [=====] - 2s 3ms/step - loss: 0.3284 - accuracy: 0.8662
Epoch 98/100
810/810 [=====] - 2s 3ms/step - loss: 0.3272 - accuracy: 0.8649
Epoch 99/100
810/810 [=====] - 2s 3ms/step - loss: 0.3306 - accuracy: 0.8648
Epoch 100/100
810/810 [=====] - 2s 2ms/step - loss: 0.3285 - accuracy: 0.8633
Out[39]: <keras.src.callbacks.History at 0x7eb540882380>

```

```

In [55]: def confusion_matrix( model, y_actual, labels=[1, 0], xtest=X_test):
y_predict = model.predict(xtest)
cm = metrics.confusion_matrix(y_actual, y_predict, labels=[0, 1])
df_cm = pd.DataFrame(cm, index=["Actual - No", "Actual - Yes"],
                      columns=['Predicted - No', 'Predicted - Yes'])

group_counts = [f"{value:0.0f}" for value in cm.flatten()]
group_percentages = [f"{value:.2%}" for value in cm.flatten()/np.sum(cm)]

labels = [f"{gc}\n{gp}" for gc, gp in zip(group_counts, group_percentages)]
labels = np.asarray(labels).reshape(2,2)

sns.heatmap(df_cm, annot=labels, fmt='')
plt.ylabel('True label', fontsize=14)
plt.xlabel('Predicted label', fontsize=14);

```

```

In [44]: y_pred_val=model.predict(X_val_num_df)
y_pred_val[y_pred_val>0.5]=1
y_pred_val[y_pred_val <0.5]=0

29/29 [=====] - 0s 1ms/step

```

```

In [45]: y_pred_val=y_pred_val.tolist()
X_compare_val=X_val.copy()
X_compare_val['y_actual']=y_val
X_compare_val['y_pred']=y_pred_val
X_compare_val.head(10)

```

Out[45]:

|  | CreditScore | Geography | Gender  | Age    | Tenure | Balance | NumOfProducts | HasCrCard | IsActiveMember | EstimatedSalary | y_actual  | y_p |   |
|--|-------------|-----------|---------|--------|--------|---------|---------------|-----------|----------------|-----------------|-----------|-----|---|
|  | 340         | 642       | Germany | Female | 40     | 6       | 129502.49     | 2         | 0              | 1               | 86099.23  | 1   | [ |
|  | 8622        | 706       | Germany | Male   | 36     | 9       | 58571.18      | 2         | 1              | 0               | 40774.01  | 0   | [ |
|  | 8401        | 535       | Spain   | Male   | 58     | 1       | 0.00          | 2         | 1              | 1               | 11779.98  | 1   | [ |
|  | 4338        | 714       | Spain   | Male   | 25     | 2       | 0.00          | 1         | 1              | 1               | 132979.43 | 0   | [ |
|  | 8915        | 606       | France  | Male   | 36     | 1       | 155655.46     | 1         | 1              | 1               | 192387.51 | 1   | [ |
|  | 2624        | 605       | Spain   | Female | 29     | 3       | 116805.82     | 1         | 0              | 0               | 4092.75   | 0   | [ |
|  | 2234        | 720       | France  | Female | 38     | 10      | 0.00          | 2         | 1              | 1               | 56229.72  | 1   | [ |
|  | 349         | 582       | France  | Male   | 39     | 5       | 0.00          | 2         | 1              | 1               | 129892.93 | 0   | [ |
|  | 3719        | 850       | France  | Female | 62     | 1       | 124678.35     | 1         | 1              | 0               | 70916.00  | 1   | [ |
|  | 2171        | 526       | Germany | Male   | 58     | 9       | 190298.89     | 2         | 1              | 1               | 191263.76 | 0   | [ |

In [48]:

```
cm_val=confusion_matrix(y_val,y_pred_val)
cm_val
```

Out[48]:

```
array([[682,  34],
       [100,  84]])
```

In [49]:

```
Accuracy=782/900
print("Accuracy of the Model on the Validation Data set is 86.89%")
```

Accuracy of the Model on the Validation Data set is 86.89%

In [50]:

```
fit_model_1 = lambda model_1, x, y, validation_data, epochs: model_1.fit(
    X_train, y_train, validation_data=validation_data, epochs=epochs
)
```

In [51]:

```
loss1,accuracy1=model.evaluate(X_train_num_df,y_train,verbose=False)
loss2,accuracy2=model.evaluate(X_val_num_df,y_val,verbose=False)
print("Train Loss {}".format(loss1))
print("Train Accuracy {}".format(accuracy1))
print("Val Loss {}".format(loss2))
print("Val Accuracy {}".format(accuracy2))
```

Train Loss 0.32871773838996887  
Train Accuracy 0.8649382591247559  
Val Loss 0.35169997811317444  
Val Accuracy 0.851111114025116

In [52]:

```
from sklearn import metrics
y_pred_test=model.predict(X_test_num_df)
y_pred_test[y_pred_test>0.5]=1
y_pred_test[y_pred_test<0.5]=0
cm_test=metrics.confusion_matrix(y_test,y_pred_test)
cm_test
print("Test Confusion Matrix")
```

32/32 [=====] - 0s 2ms/step  
Test Confusion Matrix

In [53]:

```
cm_test
```

Out[53]:

```
array([[749,  45],
       [120,  86]])
```

In [54]:

```
loss3,accuracy3=model.evaluate(X_test_num_df,y_test,verbose=False)
print("Test Accuracy is {}".format(accuracy3))
print("Test loss is {}".format(loss3))
```

Test Accuracy is 0.8349999785423279  
Test loss is 0.3771384060382843

## Actionable Insights and Recommendations

What recommendations would you suggest to the bank?

- Active members do not tend to churn. So certain cashback programs and advertisements can improve the condition.
- Customers with multiple products do not tend to churn. Adding incentives can help churn.
- Old customers tend to churn more. So, better deals should be presented to older people.